

COL106/1101 — Programming with Data Structures

Practice Programming Quiz — Set A (10 Questions)

CSE, IIT Delhi — Moodle VPL

Contents

Instructions (read before you start crying)	1
Question 1 — The Multi-Branch CampusBank	3
Question 2 — Postfix over Polynomials	5
Question 3 — The Complaint Ledger	7
Question 4 — The Unrolled Ledger	9
Question 5 — The Emergency Router	11
Question 6 — The Bounded Buffer Bakery	13
Question 7 — Skyline Renovation	15
Question 8 — The Roll Number Registry	17
Question 9 — The Ouroboros Machine	19
Question 10 — Orbital Defense Grid	21

Instructions (read before you start crying)

- This is a practice quiz assembled in the exact style of your graded VPL assignments. Each question below is calibrated for roughly **3 hours** of focused work if you were seeing it cold, so do not attempt all ten in one sitting unless you enjoy suffering.
- Every question is **Individual work**. Language: **Python 3**.
- Each question folder contains `student_code.py` (a starter file with the class/function skeletons already declared and every method body replaced with a `TODO` and a `raise NotImplementedError`), and an `autograder.py` you can run locally at any time with `python3 autograder.py` to see your score against both the visible sample tests reproduced below **and** a battery of hidden tests covering edge cases, adversarial inputs, and raw performance.
- The autograder distinguishes three failure modes, exactly like the real VPL grader: **WA** (wrong output), **TLE** (your process did not finish inside the time limit - this is almost always a sign your implementation is not asymptotically what the question demands, not that your machine is slow), and **RE** (your program crashed - an uncaught exception, bad indexing, etc.).
- Passing every visible test is necessary but nowhere near sufficient. The hidden tests are deliberately adversarial: boundary values, ties, degenerate/empty inputs, operations issued in an order designed to break an implementation that only works for the “obvious” case, and large randomised or worst-case inputs that will time out any implementation that does not

meet the stated complexity requirement. Read every docstring in the starter file as carefully as you would read the original problem statement – the exact behaviour on ties, no-ops, and boundary conditions is specified there and *is* graded.

- Unless a question says otherwise, you may not use the corresponding built-in Python data structure that the question is asking you to build from scratch (e.g. you may not implement Q8's Set ADT by wrapping Python's set). Doing so may pass the visible tests and still fail the hidden performance tests, or simply defeats the point of the exercise – either way, it will not get credit on the real thing, so don't practice the habit here.
- As in your labs: **you don't need to do any error checking** beyond what is explicitly asked for. You may assume every input respects the stated preconditions unless a test is specifically exercising what happens when a precondition is relaxed (e.g. an operation on an identifier that was never created) – and where that's possible, the question tells you the required behaviour explicitly.

Good luck. You will need it.

Question 1 — The Multi-Branch CampusBank

Topics: Object-Oriented Programming (inheritance, encapsulation, operator overloading, dunder methods)

A university is consolidating every hostel mess account, stipend disbursal account, and department petty-cash account onto one internal ledger system it calls CampusBank. Unlike a single monolithic account type, CampusBank recognises that a first-year's mess account behaves very differently from a faculty member's discretionary research account: some accounts should earn interest but be throttled against being drained by frequent withdrawals, while others should never earn interest but should be allowed to run a little negative to smooth out timing mismatches between when a grant is sanctioned and when it is actually credited. Rather than writing one bloated Account class riddled with `if self.kind == "savings"` checks scattered through every method, you are asked to model this properly with inheritance and polymorphism: a common Account base class capturing what every account has in common, and two specialised subclasses, SavingsAccount and CurrentAccount, each overriding exactly the behaviour that differs.

Every account has a unique string identifier, a holder's name, and a balance (a non-negative float at creation, though a CurrentAccount may later go negative within its overdraft limit – see below). Every account starts OPEN and may later become CLOSED (only via a MERGE, described below); once CLOSED it rejects every further operation silently. Every account also maintains an internal transaction log: `len(account)` must report exactly how many transactions have been *successfully* logged against it so far (deposits, successful withdrawals, successful incoming and outgoing legs of a transfer, and a successful incoming MERGE) – an operation that was *denied* for any reason must never increment this count, which is precisely why you cannot just increment a counter unconditionally inside deposit/withdraw and call it a day; the polymorphic withdraw methods described below must themselves decide whether an attempt succeeds before any logging happens.

A SavingsAccount earns interest and enforces a monthly withdrawal cap. It is constructed with an interest rate (a fraction such as 0.10 for 10%, applied once per TICK – see below – to a positive balance only) and an integer limit on how many *successful* withdrawals it will honour before a TICK resets the counter. Crucially, both a direct WITHDRAW **and** the outgoing leg of a TRANSFER consume one unit of this monthly allowance – a savings account that has already used up its withdrawals for the month must refuse to be the source of a transfer just as it refuses a direct withdrawal, and in both cases the account (and, for a transfer, the destination account too) must be left completely untouched, not partially updated.

A CurrentAccount never earns interest and has no withdrawal-count limit at all, but is constructed with an overdraft limit: a withdrawal (direct or as the source leg of a transfer) is denied if it would push the balance to a value *strictly less* than -overdraft – landing **exactly** on -overdraft is perfectly fine and must succeed. Getting this off-by-one wrong in either direction is exactly the kind of thing the hidden tests are built to catch.

CampusBank exposes these operations (see `student_code.py` for the exact I/O plumbing, which is already written for you – you only implement the Account/SavingsAccount/CurrentAccount/Bank classes):

```
CREATE_SAVINGS id name balance rate limit
CREATE_CURRENT id name balance overdraft
DEPOSIT id amount
WITHDRAW id amount
TRANSFER from_id to_id amount
TICK
STATUS id
TOP k
MERGE id1 id2
```

DEPOSIT/WITHDRAW/TRANSFER/TICK/MERGE produce no output and are silent no-ops whenever they target an account that was never created or is currently CLOSED (for TRANSFER, if *either* endpoint

is missing/closed, nothing happens to either side). TICK applies every open account's own tick() behaviour polymorphically (interest and a limit reset for savings accounts; nothing at all for current accounts) – notice that this is exactly the kind of place where polymorphism earns its keep: Bank.tick() should not need to know or care which concrete subclass each account is.

STATUS id prints str(account), or the literal string NOT FOUND if that id was never created at all (a CLOSED account is still “found”). The required __str__ format is "<id> <name> <TYPE> <balance:.2f> <status> <len(self)>" where <TYPE> is SAVINGS or CURRENT.

TOP k prints, one per line, the top k currently-OPEN accounts by balance descending (ties broken by account id ascending), each line formatted "<id> <balance:.2f>". This ranking **must** be produced by sorting with your own __lt__ overload (balance ascending, tie by id ascending) rather than writing a separate comparison function – the whole point of overloading __lt__ is that sorted(...) (with reverse=True here) just works.

MERGE id1 id2 is a no-op if either id is missing, either account is already CLOSED, or id1 == id2. Otherwise, the combined balance is computed using your Account.__add__ overload applied to the two account **objects** (acc1 + acc2, which – mirroring the classic “wallet” pattern you’ve seen in your OOP lab – returns the *sum of the two balances as a plain float*, not a new account object); that value, rounded, becomes id1's new balance, a "MERGE <id2>" transaction is logged on id1, and id2 becomes CLOSED with its balance zeroed out – but its transaction count as of the moment it closed is preserved forever for any later STATUS query on it.

One global rule ties the whole thing together and is enforced by every hidden test: **round every balance to exactly 2 decimal places with round(x, 2) immediately after any operation that changes it.** Two independently-correct implementations that both follow this rule will always agree bit-for-bit on every formatted balance; skip it and you will fail tests you have no way of debugging by eye, because the mismatch will be in the 10th decimal place of a number you never print in full.

Example

```
CREATE_SAVINGS A1 Priya 1000 0.10 2
CREATE_CURRENT A2 Rohan 500 200
WITHDRAW A1 100
WITHDRAW A1 100
WITHDRAW A1 100
STATUS A1
TICK
STATUS A1
```

Output:

```
A1 Priya SAVINGS 800.00 OPEN 2
A1 Priya SAVINGS 880.00 OPEN 2
```

The third WITHDRAW A1 100 is denied (the monthly limit of 2 has already been used by the first two), so it produces neither output nor a log entry, which is why len(self) still reads 2 on the very first STATUS. The TICK then applies 10% interest to the surviving 800.00 balance, producing 880.00, and separately resets the monthly withdrawal counter (not observable here, since no further WITHDRAW follows).

Requested files

student_code.py — implement Account, SavingsAccount, CurrentAccount, and Bank exactly as specified; do not rename any class or public method, since the autograder drives your code only through Bank's public interface (via the already-written solve() at the bottom of the file).

Question 2 — Postfix over Polynomials

Topics: Functional programming style (reduce, closures, immutability, recursion)

You already know how to evaluate a postfix arithmetic expression with a stack: scan the tokens left to right, push operands, and apply each operator to whatever sits on top of the stack. Last week you saw that this entire left-to-right scan is nothing but a single reduce over the token stream, threading an (immutable, always-replaced-not-mutated) stack through one combining step per token. This week you generalise the *values* the stack can hold from plain integers to something richer: **polynomials**.

A polynomial with integer coefficients is represented, throughout this question, as an **immutable tuple** ($c_0, c_1, c_2, \dots$) meaning the function $c_0 + c_1x + c_2x^2 + \dots$, lowest degree first, with no trailing zero coefficients (so the zero polynomial is the empty tuple $()$). A “value” that can sit on the stack is therefore either a plain int or one of these tuples, and you must be able to tell the two apart at runtime (`is_poly`, which you are given, does exactly that with `isinstance(v, tuple)`).

Six operators act on these values, and every one of them pops its operands off the top of the stack and pushes back exactly one result, exactly like the arithmetic operators you already know from ordinary postfix expressions:

- `+: poly poly -> poly` (ordinary polynomial addition).
- `-: poly poly -> poly`, where `a b -` computes $a - b$ (subtraction is **not** commutative, and the autograder specifically checks that you have the operand order the right way around).
- `scale: poly k -> poly`, multiplying every coefficient by the integer `k` (so `a k scale` computes $k \cdot a$).
- `deriv: poly -> poly`, the formal derivative ($\frac{d}{dx}(c_i x^i) = i \cdot c_i x^{i-1}$; the derivative of a constant, or of the zero polynomial, is the zero polynomial).
- `integ: poly -> poly`, the antiderivative with constant term fixed at zero (so `c0 + c1 x + ...` becomes `0 + c0 x + (c1/2) x^2 + (c2/3) x^3 + ...`); wherever a division comes out even, keep it as an exact int, and only fall back to a float when it genuinely doesn't (a helper `_rat` is given to you that does exactly this for one division – use it once per coefficient).
- `eval: poly x -> int/float`, evaluating the polynomial at the integer `x` (so `a x eval` computes $a(x)$).

On top of the operators, you support **named words**: a word is bound, in a words registry supplied alongside the program, to a list of tokens that is simply *replayed against the current stack* whenever the word's name appears, with exactly the effect those tokens would have had if they had been written inline in that spot. For instance if `words = {"second_deriv": ["deriv", "deriv"]}`, then the token sequence `["f", "second_deriv"]` must behave identically to `["f", "deriv", "deriv"]`, and a word may itself invoke other words (so you might further define `"fourth_deriv": ["second_deriv", "second_deriv"]`). You are **not** told the mechanism for handling a word token up front – work it out from the observation that “run a sequence of tokens over a stack” is *precisely* what your whole evaluator already does for the top-level program, so the word case should fall directly out of machinery you already have (specifically: building another instance of your own combiner for the word's own body, and `reduce_-ing` it over that body, starting from the current stack).

Hard rules, enforced both by the autograder and by manual review of your submission, exactly as in your original postfix lab:

- All iteration must go through your own `reduce_`, **written with recursion** – no `for/while` loop may replace it as the driver of token processing, and no helper (`poly_add`, `poly_scale`, ...) may use a `for/while` loop internally either; use recursion there too.
- **Nothing is ever mutated**. Every helper returns a brand-new tuple; the ones passed in are left completely untouched, and no name inside a function may be reassigned once it has a value.
- You may not use `eval()`, `exec()`, or any other built-in expression evaluator to shortcut any part of this.

Example

Take polynomials $f = x^2$ (given as `f 0 0 1`) and $g = x$ (given as `g 0 1`), and one word `double = [2, "scale"]`.

```
f g +      -> [0,1,1]      (x^2 + x)
f deriv    -> [0,2]        (2x)
f 3 eval   -> 9            (f(3) = 9)
g integ    -> [0,0,0.5]    (x^2 / 2)
f double   -> [0,0,2]      (2x^2)
```

Input / Output format

```
<npolys>
<npolys lines: name c0 c1 c2 ...>
<nwords>
<nwords lines: wordname tok1 tok2 ...>
<q>
<q lines: a whole postfix program, space-separated tokens>
```

For each of the `q` query lines, print the resulting value on its own line: a bare integer/float via `str(...)` for a plain number, or `[c0,c1,c2,...]` (no spaces, `[]` for the zero polynomial) for a polynomial. This formatting function (`fmt_value`) is given to you – do not change it, just make sure `evaluate_postfix` returns the right underlying value.

Requested files

student_code.py — implement `reduce_`, `poly_add`, `poly_neg`, `poly_sub`, `poly_scale`, `poly_deriv`, `poly_integ`, `poly_eval`, `step`, and `evaluate_postfix` exactly as specified.

Question 3 — The Complaint Ledger

Topics: Asymptotic analysis, arrays, amortized complexity

The campus facilities office logs every maintenance complaint's resolution time (in hours) as it comes in, and wants a live dashboard showing the average resolution time over the last k complaints, for whatever k a staff member types in. You are *not* allowed to reach for Python's `list.append/list.pop` and call it a day – your actual task is to build the dynamic array underneath from first principles, exactly the way `list.append` is amortised $O(1)$ under the hood, and then build the moving-average feature on top of it.

Part A — Vector: a dynamic array from scratch

Implement Vector using, as its **only** storage, a single fixed-length Python list treated as a raw, fixed-capacity array: you manually track how many slots are actually in use (`_size`) versus how many exist (`_cap`), and you manually reallocate (copying element by element, by index – not via a slice that secretly does the copy for you) whenever the policy below calls for it. `push_back`, `pop_back`, `get`, `set`, `size`, and `capacity` are the only operations, and every one of them must be $O(1)$ **amortised** (get/set/size/capacity are in fact $O(1)$ *worst case*; the amortisation only matters for `push_back/pop_back`).

The capacity policy, followed exactly, because CAPACITY is graded output, not just an internal implementation detail:

- Initial capacity is **1**.
- `push_back`: if the array is completely full (`size == capacity`) *before* inserting the new element, first double the capacity (reallocating and copying every existing element across), **then** insert.
- `pop_back`: remove the last element *first*; **then**, only if the resulting size is strictly positive **and** that size is $\leq \text{capacity} // 4$, halve the capacity (never below 1), reallocating and copying the survivors across. If the size becomes exactly 0, do **not** shrink any further, no matter how large the capacity currently is (this asymmetric guard is what prevents an empty-then-refill cycle from thrashing the array back and forth between two capacities on every single operation).

Worked trace, starting from an empty Vector (capacity 1, size 0):

```
push_back(5)    -> full (size 0 == cap 1) -> grow to cap 2 -> insert -> cap=2 size=1
push_back(7)    -> not full (1 != 2)      -> insert          -> cap=2 size=2
push_back(9)    -> full (2 == 2)          -> grow to cap 4 -> insert -> cap=4 size=3
push_back(1)    -> not full (3 != 4)      -> insert          -> cap=4 size=4
pop_back()      -> remove -> size=3; 3 <= 4//4=1? no        -> cap stays 4
pop_back()      -> remove -> size=2; 2 <= 1? no             -> cap stays 4
pop_back()      -> remove -> size=1; 1 <= 1? yes -> shrink to cap=max(1,4//2)=2
```

Part B — MovingAverageTracker

Wrap a Vector of raw resolution times with a *second* Vector, `prefix`, where `prefix.get(i)` holds the sum of the first i entries (so `prefix.get(0) == 0` and `prefix.get(n)` is the grand total). This is what lets `average_last(k)` answer in $O(1)$ – a single subtraction of two prefix sums followed by one division – instead of re-summing the last k raw entries on every query, which would make a long run of large-window queries quadratic overall. `add/undo` extend or shrink both Vectors in lockstep; `set_` (updating one historical entry) is allowed to cost $O(n)$, since it must propagate the change through every later prefix sum, and that is *not* the operation whose complexity is being examined here.

Commands

PUSH x -- add x to the ledger
POP -- remove the most recent entry
GET i -- report the entry at position i
SET i x -- overwrite the entry at position i
SIZE
CAPACITY -- capacity of the MAIN ledger Vector
AVG k -- average of the last k entries; "NA" if k<=0 or k>size

AVG output: a whole-number average is printed as a bare integer (no decimal point); otherwise print it rounded to exactly 4 decimal places.

Example

PUSH 5
PUSH 7
PUSH 9
CAPACITY
PUSH 1
CAPACITY
POP
POP
POP
CAPACITY
AVG 2
SIZE
GET 0

Output:

4
4
2
NA
1
5

Requested files

student_code.py — implement Vector and MovingAverageTracker exactly as specified. A stress test pushing and popping hundreds of thousands of elements, and a separate stress test issuing thousands of full-window AVG queries against a large ledger, are both part of the hidden test suite specifically to catch an implementation that is correct but not actually amortised $O(1)$ per operation.

Question 4 — The Unrolled Ledger

Topics: Linked lists, ADT design, sqrt-decomposition

A plain singly (or doubly) linked list gives you $O(1)$ append/pop at the ends, but paying for it with $O(n)$ indexed access is exactly the opposite trade-off of a plain array. This question asks you to build a structure that gets (most of) the best of both worlds: an **unrolled linked list**, in which the sequence is chopped into fixed-size *blocks*, and the blocks themselves – not the individual elements – are what gets linked together.

Concretely: UnrolledList is a doubly linked list of Block objects. Each Block holds up to BLOCK_CAP elements (you may store a block's own elements in a plain Python list – the point of this exercise is the block-to-block linking, not reinventing array storage a second time within a block) plus prev/next pointers to its neighbouring blocks. There is **no** outer Python list of blocks anywhere in a correct solution – blocks are chained purely through prev/next.

- **append(x)**: if the tail block is full ($\text{len}(\text{tail.items}) == \text{BLOCK_CAP}$), first create a brand-new block, link it in after the current tail, and make it the new tail – *then* place x there.
- **pop()**: remove the last element of the tail block. If that empties the tail block *and* there is a previous block, drop the now-empty block from the chain and make the previous block the new tail (but if the whole list has only ever had one block, keep it even when empty, so the next append has somewhere to land).
- **get(i) / set(i, x)**: locate the block containing logical index i by *walking the block chain* – never by flattening the structure first. For a bit of extra speed (and because the list is doubly linked, so there's no reason not to), walk from whichever end, head or tail, is closer to i .
- **__len__**: $O(1)$, via a maintained running counter.

A block-chain walk visits roughly $\text{size} / \text{BLOCK_CAP}$ blocks rather than size individual nodes – for BLOCK_CAP in the hundreds, that is a dramatic constant-factor win over a naive one-node-per-element linked list on any workload that needs indexed access, which is exactly what the performance tests below are built to expose. (This is a simplified, fixed-block-size relative of full sqrt-decomposition with adaptive rebuilding; correctness never depends on the exact value of BLOCK_CAP, only performance does, so you are free to tune it.)

Commands

Identical in spirit to a sequence ADT you've built before:

```
append x
pop
access i
set i x
len
```

append/set/pop produce no output; access i prints the element at position i ; pop prints the removed element; len prints the current size.

Example

```
append 5
append 3
append 8
append 1
pop
access 1
append 9
append 2
len
```

```
pop
pop
access 0
set 0 100
access 0
```

Output:

```
1
3
5
2
9
5
100
```

Requested files

student_code.py — implement `Block` and `UnrolledList` exactly as specified. The hidden performance tests build lists of tens to hundreds of thousands of elements and then hammer them with thousands of far-apart indexed accesses and near-tail `set/access` pairs; an implementation that secretly degrades to a one-node-per-element walk (or that always walks from the head regardless of which end is closer) will time out where a genuine block-chain walk finishes in well under a second.

Question 5 — The Emergency Router

Topics: ADT combination — queues, a stack, and a hash map, all built from linked structures

A hospital's emergency department triages every patient into one of **five** urgency levels (1 = most urgent, 5 = least), and within a level, patients are seen strictly in the order they arrived - a textbook multi-level priority queue, except this hospital also needs to (a) move an already-waiting patient to a different level without disturbing anyone else's position, and (b) undo the most recent discharge if it turns out to have been premature. Neither of those two features is expressible with "just a priority queue"; you need a queue *combined with* a hash map (for instant lookup) and *combined with* a stack (for the undo history), all working together, and every single operation below must run in $O(1)$.

Represent each of the five levels as its own doubly linked FIFO (LevelQueue), and maintain one global dict mapping a patient's id to the exact node object currently holding them within whichever level's queue they're in. This is what makes REQUEUE possible in $O(1)$: rather than scanning a level's queue looking for the patient (which would be $O(\text{level size})$, and which the hidden performance tests are specifically built to punish), you jump straight to their node via the hash map and unlink it using only that node's own prev/next pointers.

```
REGISTER pid level    -- new patient, back of that level's queue
NEXT                  -- id of the patient at the front of the lowest-
                        numbered non-empty level, without removing
                        them, or -1 if nobody is waiting
SERVE                  -- like NEXT, but removes that patient and pushes
                        (pid, their level) onto a discharge STACK, for
                        RECALL to undo later; -1 if nobody is waiting
COUNT                -- how many patients are currently waiting, in
                        total, across every level
REQUEUE pid level     -- move an ALREADY-WAITING patient to a different
                        level's queue (at the BACK of the new level).
                        No-op if pid is not currently waiting.
RECALL                 -- pop the most recently discharged patient off
                        the discharge stack and put them back, at the
                        FRONT (not the back) of the queue for their
                        ORIGINAL level -- the level they were served
                        from, not any level they may have been
                        REQUEUED through earlier while still waiting
                        (REQUEUE only ever affects patients who are
                        still waiting, so this distinction is always
                        well-defined). -1 if the discharge stack is
                        empty.
```

A subtlety the hidden tests specifically probe: a patient id may be REGISTERed again after having been fully SERVED (they are, after all, a returning patient) - your hash map lookup must correctly reflect *only* whoever is currently waiting under that id, so a stale mapping from a previous visit must never leak into a later one. Similarly, RECALL can be invoked several times in a row: since it pops a genuine stack, the recalls must come back out in strict last-discharged-first order (LIFO), even when several patients were discharged back to back from different levels.

Example

```
REGISTER p1 3
REGISTER p2 1
REGISTER p3 1
NEXT
```

```
SERVE
NEXT
REQUEUE p3 5
NEXT
SERVE
RECALL
COUNT
NEXT
```

Output:

```
p2
p2
p3
p1
p1
p1
2
p1
```

Trace: p2 and p3 both sit at level 1 (p2 first), p1 at level 3, so NEXT reports p2; SERVE removes and returns p2; NEXT now reports p3 (the only one left at level 1); REQUEUE p3 5 moves p3 down to level 5, so NEXT now reports p1 (level 3 is now the lowest non-empty level); SERVE removes p1, pushing (p1, 3) onto the discharge stack; RECALL pops that and reinserts p1 at the *front* of level 3's queue; COUNT is 2 (p1 and p3 are both waiting again); NEXT reports p1.

Requested files

student_code.py — implement PNode, LevelQueue, and Router exactly as specified. A hidden performance test registers 100,000 patients, issues 100,000 REQUEUEs against randomly chosen existing patients, and then drains everyone with SERVE – an implementation whose REQUEUE scans a level's queue to find its target will not finish in time.

Question 6 — The Bounded Buffer Bakery

Topics: Concurrency (producer/consumer, mutual exclusion, fairness), queues

This question asks you to *simulate*, by hand and completely deterministically, the classic bounded-buffer producer/consumer problem that a semaphore and a mutex would otherwise enforce for you at the OS level. There are no real threads here – that would make the output different on every run, which is useless for grading – instead, every PRODUCE/CONSUME attempt is processed strictly in the order it appears in the input, and you are responsible for reproducing exactly the blocking, waking, and fairness behaviour a correct semaphore-based implementation would exhibit.

There is one shared circular Buffer of fixed capacity C (a fixed-size array with wraparound indices – the same circular-array technique you’d use to implement a bounded queue directly on an array, without ever shifting existing elements). Threads (arbitrary string ids) attempt operations one at a time:

- **PRODUCE tid item**: if there is space in the buffer **and** no other producer is already waiting (fairness: nobody may cut in line ahead of an earlier-blocked producer, even if space happens to be available at this exact moment), the item is inserted immediately (PRODUCED tid item). Otherwise tid joins the back of the producer wait-queue (BLOCKED tid). Immediately after any successful insertion, check whether the front of the *consumer* wait-queue can now be served (see “waking” below).
- **CONSUME tid**: symmetric. If the buffer is non-empty **and** no other consumer is already waiting, remove the front item immediately (CONSUMED tid item); otherwise tid blocks (BLOCKED tid). Immediately after any successful removal, check whether the front of the *producer* wait-queue can now be let in.
- **Waking**: whenever the buffer gains room (a successful CONSUME), let waiting producers in, strictly in the order they blocked, for as long as room remains – for each one, log WOKEN tid PRODUCED item (using the item *that producer originally tried to insert* when they first blocked – you must remember it alongside their id). Whenever the buffer gains an item (a successful PRODUCE, including one that was itself just woken up), symmetrically wake waiting consumers, logging WOKEN tid CONSUMED item for each.
- **STATUS**: prints the buffer’s current contents front-to-back and the id at the front of each wait queue (- if empty); produces no log entries of its own.

Performance requirement, not just a style preference: both wait queues must support $O(1)$ push-to-back and $O(1)$ pop-from-front. Using a Python list with `list.pop(0)` is $O(n)$ per call and will not survive the hidden performance tests – build your own linked-list-based queue, exactly like the ones you’ve already built elsewhere in this course.

Example (capacity 2)

```
PRODUCE t1 A
PRODUCE t2 B
PRODUCE t3 C
STATUS
CONSUME c1
STATUS
CONSUME c2
CONSUME c3
STATUS
```

Output:

```
PRODUCED t1 A
PRODUCED t2 B
BLOCKED t3
BUF[A,B] PWAIT=t3 CWAIT=-
```

```
CONSUMED c1 A
WOKEN t3 PRODUCED C
BUF[B,C] PWAIT=- CWAIT=-
CONSUMED c2 B
CONSUMED c3 C
BUF[] PWAIT=- CWAIT=-
```

Note how CONSUME c1 produces *two* log lines: the consumption itself, and the immediate wake-up of t3, who had been waiting since the buffer first filled up.

Requested files

student_code.py — implement Buffer, WaitQueue, and Bakery exactly as specified. Two hidden performance tests each queue up hundreds of thousands of threads on one side of a capacity-1 buffer and then drain them one at a time from the other side – this is precisely the access pattern that turns an $O(n)$ -per-pop wait queue into an unusable $O(n^2)$ simulation.

Question 7 — Skyline Renovation

Topics: Stack ADT and its classic applications, built entirely on a stack you write yourself

A city planning tool reveals a skyline of building heights one building at a time (new buildings are only ever appended on the right, and the most recent one can be undone), and must answer three questions about whatever skyline currently exists – each of which is a famous application of a **monotonic stack**, and each of which you must implement using your **own** Stack class (array-backed, with push/pop/peek/is_empty/size) rather than treating a bare Python list as a stack directly inside Skyline’s own methods.

- **SPAN *i*** – the “stock span” of building *i*: the length of the longest run of consecutive buildings ending at (and including) index *i*, reading leftward, all of whose heights are $\leq \text{heights}[i]$ (a single taller building anywhere in that run stops the count). This must be answered using the span value that was computed **at the moment building *i* was appended** – not recomputed from scratch on every query.
- **NEXTGREATER *i*** – the smallest index $j > i$ with $\text{heights}[j]$ *strictly* greater than $\text{heights}[i]$ (an equal height does not count), or -1 if no such *j* currently exists.
- **MAXRECT** – the area of the largest axis-aligned rectangle that fits under the current skyline (the classic “largest rectangle in a histogram” problem).

Complexity split, deliberately asymmetric: APPEND must run in **amortised** $O(1)$, by maintaining a monotonically decreasing-height stack of indices *incrementally* – exactly the single-pass algorithm you’d use to compute every building’s span in one left-to-right sweep, except here you’re doing it one APPEND at a time as buildings stream in, popping off (and *summing the span values of*) every index whose height is $\leq$ the new building’s height before pushing the new index on top. (Think carefully about *why* summing the popped spans, rather than just counting how many indices got popped, gives the right answer – this is the crux of the whole algorithm.) NEXTGREATER and MAXRECT, by contrast, **may** be recomputed from scratch on every call, in $O(\text{current number of buildings})$, using a fresh monotonic-stack scan each time – that is intended and expected, not a shortcut you’re getting away with. REMOVE_LAST **may** also cost $O(\text{current } n)$: rebuilding your incremental span bookkeeping from scratch after an undo is completely acceptable (getting an $O(1)$ undo working is a much harder problem this question does not ask you to solve).

Example

```
APPEND 2
APPEND 1
APPEND 5
APPEND 6
APPEND 2
APPEND 3
SPAN 0
SPAN 1
SPAN 2
SPAN 3
SPAN 4
SPAN 5
MAXRECT
NEXTGREATER 2
```

Output:

```
1
1
3
4
```

1
2
10
3

Trace for SPAN 2 (height 5): looking left from index 2, heights are 1 (index 1, ≤ 5 , keep going), then 2 (index 0, ≤ 5 , keep going) – the run reaches the start, giving span 3. MAXRECT on [2, 1, 5, 6, 2, 3] is 10 (the two adjacent bars of height 5 and 6 together, width 2, height $\min(5, 6) = 5$). NEXTGREATER 2 (height 5, at index 2) looks rightward for the first strictly taller bar: index 3 (height 6) qualifies.

Requested files

student_code.py — implement Stack and Skyline exactly as specified. A hidden performance test appends 40,000 buildings (with periodic SPAN queries interleaved) specifically to catch an APPEND that rescans everything instead of maintaining its stack incrementally; a second hidden test issues hundreds of MAXRECT queries against a few thousand buildings to catch an accidentally-quadratic largest-rectangle scan.

Question 8 — The Roll Number Registry

Topics: Set ADT, hash tables, open addressing, amortized analysis

Build HashSet completely from scratch using **open addressing** with **quadratic (triangular-number) probing** and **tombstones** for deletion. You may not use Python's built-in set, frozenset, or dict as the underlying storage – the only allowed backing store is a single fixed-size Python list of “slots”, each of which holds either an actual element, or one of two given sentinel objects: FREE (never used) or TOMB (something used to be here, but was removed – keep probing past it, but it's still “occupied” for load-factor purposes until the next resize reclaims the space).

The probe sequence (given to you, do not change the formula) is

$$\text{idx}_i = (\text{hash}(x) + \frac{i(i+1)}{2}) \bmod \text{capacity}, \quad i = 0, 1, 2, \dots$$

This particular triangular-number formula is guaranteed to visit every slot of the table exactly once before repeating, **as long as the capacity is a power of two** – which is precisely why the initial capacity is fixed at 8 and every resize below multiplies or divides the capacity by exactly 2.

Resizing policy, followed exactly (this is what makes add/remove amortised $O(1)$, so it is graded, not merely suggested):

- After a successful `add()` pushes the number of **occupied** slots (live elements *and* tombstones together) past **70%** of capacity, double the capacity and rehash every currently-*live* element into a fresh table – this rehash is also how tombstone space gets reclaimed, since tombstones are simply dropped during a resize rather than copied forward.
- After a successful `remove()` drops the number of **live** elements below **20%** of capacity, halve the capacity (never below the initial 8) and rehash the same way.

```
CREATE name          -- explicitly create an empty named set
                      (sets are also created implicitly the
                      first time any other command names them)

ADD name x
REMOVE name x
CONTAINS name x      -- prints YES or NO
SIZE name
ITEMS name           -- prints every live element, comma
                      separated, SORTED using key=str (so
                      mixed int/string elements compare
                      consistently)

UNION name1 name2 dest
INTERSECT name1 name2 dest
DIFF name1 name2 dest -- elements in name1 but not in name2
```

`x` is parsed as an int when possible, otherwise treated as a plain string, so a registry can freely mix numeric roll numbers with string identifiers. `UNION/INTERSECT/DIFF` build a brand new HashSet and store it under `dest` (which may legally be the same name as one of the two operands – your implementation must not corrupt the source set while it is still being read from, so build the destination separately and only bind it to the registry once it's complete).

Example

```
CREATE A
ADD A 5
ADD A 3
ADD A 5
CONTAINS A 5
CONTAINS A 9
```

```
SIZE A
REMOVE A 3
SIZE A
CREATE B
ADD B 3
ADD B 5
ADD B 7
UNION A B U
ITEMS U
INTERSECT A B I
ITEMS I
```

Output:

```
YES
NO
2
1
3,5,7
5
```

(ADD A 5 a second time is a no-op, which is why SIZE A reads 2, not 3, right after the three ADDs.)

Requested files

student_code.py — implement HashSet exactly as specified. A hidden performance test adds 200,000 distinct keys and then issues 100,000 CONTAINS lookups; a second churns 150,000 interleaved ADD/REMOVE calls against a small key range specifically to stress the grow/shrink resize policy (and tombstone reclamation) under sustained load.

Question 9 — The Ouroboros Machine

Topics: Linear algebra as a program-analysis tool, matrix exponentiation, stack-based interpretation of nested loops

A toy assembly-like language operates on exactly **6 registers**, r_0 through r_5 , each holding an integer modulo a fixed prime-ish modulus $p \geq 2$. There are three statement types (i, j are register indices 0-5):

```
add ri rj    -- ri <- (ri + rj) mod p      (rj unchanged)
sub ri rj    -- ri <- (ri - rj) mod p      (rj unchanged)
swap ri rj   -- ri and rj trade values    (both change)
```

together with a block construct, which may be nested arbitrarily deeply:

```
loop k
...
end
```

running everything between `loop k` and its matching `end` exactly k times, in order (k is a non-negative integer, and may be astronomically large – see below).

Exactly as you discovered in your earlier lab on a language over priority-queue-like variables, **every statement here is a linear function of the register vector** – yes, even `swap`, which is simply a permutation matrix – so the *entire program*, regardless of how deeply its loops nest, corresponds to a single 6×6 matrix M such that running the program once is equivalent to left-multiplying the initial register vector $(r_0, \dots, r_5)^\top$ by M . Your job is to compute and return that matrix, every entry reduced into $[0, p-1]$.

The detail everyone gets backwards: if statement T_1 runs before statement T_2 , then running both on a vector v gives $T_2(T_1 v) = (T_2 T_1) v$ – so the net matrix accumulates with each *new* statement’s matrix multiplied **on the left** of whatever came before it, never on the right. A program that only ever gets this backwards will still pass a same-single-statement sample test by accident and then fail almost everything else.

Efficiency: `loop k ... end` must be handled with **fast matrix exponentiation** (repeated squaring, $O(\log k)$ matrix multiplications) applied to the block’s own net matrix – never by literally executing the body k times, since k may be as large as 10^{15} and nesting means the *effective* number of iterations can be astronomically larger still even when the program text itself is tiny. Maintain **two stacks** as you scan the program: one of matrix accumulators (one per currently-open block, including the outermost “whole program” block, which starts as the identity matrix), and one of pending loop counts. On `add/sub/swap`, build that statement’s own elementary matrix, pop the current (innermost) accumulator, and push back $(E @ \text{accumulator}) \bmod p$. On `loop k`, push a fresh identity accumulator and push k onto the count stack. On `end`, pop the just-finished block’s accumulator (body), pop the matching count k , compute $\text{body}^k \bmod p$ via fast exponentiation, pop the now-exposed enclosing accumulator (outer), and push back $(\text{body}^k @ \text{outer}) \bmod p$ – folding the *entire loop*, as a single unit, into its enclosing block using exactly the same left-multiplication rule as any ordinary statement.

Example

Program:

```
add r0 r1
loop 2
add r1 r2
end
add r2 r0
```

with $p = 1000000007$. Tracking r_0, r_1, r_2 symbolically from initial values a, b, c (registers $r3..r5$ never touched, so their rows/columns of M stay exactly the identity): after `add r0 r1`, $r_0 = a + b$. The loop runs `add r1 r2` twice, so $r_1 = b + 2c$ ($r2$ is never modified inside the loop, so each of the two iterations adds the *same*, unchanged $r2$). Finally `add r2 r0` uses the *already-updated* $r0$, giving $r_2 = c + (a + b)$. So the matrix's first three rows read (in $a, b, c, \dots$ order):

```
1 1 0 0 0 0
0 1 2 0 0 0
1 1 1 0 0 0
```

and rows 4-6 are exactly the identity.

Input / Output format

<p>

<the program: one statement per line, loop/end properly nested and closed, running to the end of the input>

Output: the 6×6 matrix, one row per line, six space-separated integers each in $[0, p-1]$.

Requested files

student_code.py — implement `matpow`, `elem_add`, `elem_sub`, `elem_swap`, and `evaluate_program` exactly as specified (identity and `matmul` are given to you, fully working, for you to build on top of). One hidden performance test uses a single loop `1000000000000000000` around one statement; another nests 30 `loop 2` blocks around a single inner statement, so the *effective* iteration count is 2^{30} while the program text itself is only 61 lines long – only genuine matrix exponentiation, never literal unrolling at any level, survives either one.

Question 10 — Orbital Defense Grid

Topics: Continuous-time simulation, numerical root-finding, priority of simultaneous events (the hardest question on this paper)

A square arena stretches from $-B$ to $+B$ on both axes. Drones move in 2D under **piecewise-constant acceleration** - exactly as in your Air Traffic Control lab: a drone holds whatever acceleration it was last given (starting at $(0,0)$) until told otherwise, so between instructions its position and velocity obey

$$p(t) = p_0 + v_0(t - t_0) + \frac{1}{2}a(t - t_0)^2, \quad v(t) = v_0 + a(t - t_0).$$

A drone is **instantly and permanently destroyed** - its status becomes accident from that exact instant on, it stops moving forever, and it can never again be involved in any further event, including a collision that would otherwise have happened a moment later - the first instant **any** of these three conditions holds, checked **continuously**, not merely at instruction boundaries:

1. **Boundary breach:** $|x(t)| \geq B$ or $|y(t)| \geq B$ (touching a wall counts, even if the trajectory would otherwise have curved back inside a moment later - there is no tangency exemption in this arena).
2. **Overspeed:** its speed strictly exceeds its own `max_speed`.
3. **Collision:** its distance to some *other currently-surviving* drone becomes $\leq$ the sum of their two radii.

Because two of these can become true at the exact same instant, and because a drone that has just been destroyed is no longer *there* to be hit, you must resolve events in strict chronological order with one tie-breaking rule, carried straight over from your ATC lab: whenever several destructions coincide at the same instant, **every self-destruction (boundary breach or overspeed) at that instant is applied before any collision at that same instant is even considered**, so a drone that self-destructs at time t can never simultaneously be credited with a collision at that same t .

Use the supplied `is_equal/is_less/is_greater/is_less_equal/is_greater_equal` helpers (EPS = $1e-6$) for **every** comparison involving a computed float - never write `x == y` or `x > y` directly. The test data guarantees that any two mathematically distinct event times whose relative order could affect the answer differ by at least 10^{-3} .

The genuinely hard part: in your ATC lab, each condition was a single quadratic in time, solvable in closed form. Here, boundary breach and overspeed are *still* single-drone quadratics (a working `quadratic_roots` helper is given to you) - but the distance between **two** independently accelerating drones is a **quartic** (degree-4) polynomial in time, and quartics do not have a clean formula you're expected to hand-derive. Instead, find the collision instant **numerically**: express both drones' relative position/velocity as of whichever of them was more recently given an instruction, then scan the relevant time window in several hundred small steps, evaluating $f(t) = |\text{relative position}(t)|^2 - (r_1 + r_2)^2$; the moment this crosses from positive to non-positive between two consecutive samples, **bisect** within that bracket (about 100 iterations is plenty) to refine the crossing to well within EPS.

```
CREATE t id max_speed radius x y vx vy
UPDATE t id ax ay
STATUS t id          -- prints "flying", "accident", or "does not
                      exist" if that id was never created
```

Example

```
CREATE 0 D1 3 0.5 0 0 0 0
UPDATE 0 D1 2 0
STATUS 1 D1
```

STATUS 2 D1

Output:

flying
accident

D1 starts at rest at the origin and is told to accelerate at $(2, 0)$; its speed at time t is $2t$, which strictly exceeds its `max_speed` of 3 as soon as $t > 1.5$ - so it is still flying at $t = 1$ but has already suffered an accident by $t = 2$.

A second worked case, showing the self-destruction-before-collision rule in action: SG55 (max speed 3) is given acceleration $(2, 0)$ from rest at the origin and would, if it survived, collide with SG56 (cruising toward it at constant velocity from $x = 8$) at $t = 2$ - but SG55 overspeeds and is destroyed at $t = 1.5$, strictly before that would-be collision, so SG56 sails straight through the collision point completely undisturbed and is still flying well afterward.

Requested files

student_code.py — implement Drone and Arena exactly as specified (`quadratic_roots` and the `is_*` comparison helpers are given to you, fully working). This question deliberately does **not** carry a large-scale performance requirement - correctness under continuous time, with all three destruction conditions and their exact priority, is already hard enough; the hidden tests instead throw adversarial *timing* at you: drones created already touching a wall, instructions sent to a drone that (unknown to whoever sent it) had already met its end moments earlier, a later acceleration that curves a drone out of an apparently-certain collision course, and a modest multi-drone randomised scenario mixing all three destruction conditions at once.